

# Optimisation

## Contents

|          |                                      |          |
|----------|--------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                  | <b>1</b> |
| <b>2</b> | <b>Optimisation CPU</b>              | <b>1</b> |
| <b>3</b> | <b>Optimisation GPU</b>              | <b>1</b> |
| <b>4</b> | <b>Optimisation des ressources</b>   | <b>1</b> |
| <b>5</b> | <b>Profiling de base</b>             | <b>2</b> |
| <b>6</b> | <b>Optimisation Unreal</b>           | <b>2</b> |
| 6.1      | Profiling general . . . . .          | 2        |
| 6.2      | Stat profiler . . . . .              | 3        |
| 6.3      | Profiler UE5 (Insight) . . . . .     | 3        |
| 6.4      | GPU Bottleneck . . . . .             | 3        |
| 6.5      | CPU Bottleneck . . . . .             | 4        |
| 6.6      | Bonnes pratiques . . . . .           | 4        |
| 6.7      | Multithreading . . . . .             | 4        |
| 6.8      | Techniques d'optimisations . . . . . | 5        |
| 6.8.1    | Frustum Culling . . . . .            | 5        |
| 6.8.2    | Occlusion Culling . . . . .          | 5        |
| 6.8.3    | Distance Culling . . . . .           | 5        |
| 6.8.4    | LOD . . . . .                        | 5        |
| 6.8.5    | Level Streaming . . . . .            | 5        |
| 6.8.6    | Mise en cache PSO . . . . .          | 5        |
| 6.8.7    | Light complexity . . . . .           | 5        |
| 6.8.8    | Shader complexity . . . . .          | 6        |
| 6.8.9    | Overdraw . . . . .                   | 6        |
| 6.9      | Render Doc . . . . .                 | 6        |

## 1 Introduction

Ce document donnera des explications sur les bases de l'optimisation logicielle et Unreal engine. L'optimisation est un domaine très vaste de la programmation, qu'il est important de garder en tête à tous les niveaux mais dont il ne faut pas abuser inutilement. En l'occurrence on essaye d'abord de construire une structure de code solide et modulaire, qui sera ensuite plus simple à optimiser quand ce sera réellement nécessaire, et ce de manière ciblée.

Important de noter que le niveau d'optimisation est déterminé en fonction des besoins et exigences du projet, en général les frame budgets qu'on retrouve sont dans ces eaux là

— FPS — Temps max —

— 60 FPS — 16.66 ms —

— 30 FPS — 33.33 ms —

— 120 FPS — 8.33 ms —

## 2 Optimisation CPU

Ici on cherche à limiter le temps de calcul/exécution de nos instructions, on privilégiera notamment les event aux tick/update, on peut aussi appliquer l'early exit ou plus globalement mieux gérer nos boucles.

Pour éviter les cache miss (quand la donnée n'est pas trouvée en mémoire dans la même "section") on privilégie les Array aux liste/vector. Opter pour du data oriented peut aussi être intéressant. Limiter les allocations dynamiques de mémoire, utiliser le pooling quand c'est possible.

## 3 Optimisation GPU

Pour ce qui est du GPU le principal défi sera les draw calls (appel cpu au gpu pour afficher un état de rendu), peu de draw call = charge gpu mais trop de draw call = charge cpu.

Pour se faire il est plus délicat d'agir par le code, on jouera davantage sur les assets graphiques, le code shader ou les batches d'objets partageant une texture.

## 4 Optimisation des ressources

Comme dit plus tôt on peut jouer sur les assets pour optimiser notre jeu, les ressources graphiques mais pas que.

Pour les textures on :

- utilise des dimensions en puissance 2
- utilise des atlas/texture groups
- active les mip map

Pour les mesh on :

- limite le nombre de polygones
- active le LOD

Pour la physique on :

- rends pas tout physique
- met des objets en static
- simplifie les collider
- dose les requete au moteur physique (raycast, overlap)
- filtre avec des layers

Pour le reseau on :

- serialise les paquets
- limite la fréquence d'envoi
- envoi que les infos importantes (pas les vfx!!!)

## 5 Profiling de base

Premiere chose à faire. On identifie les bottleneck en : chronometrant le temps passé dans les fonctions, utilisant des tools de profiling mémoire (profler visual, valgrind), de profiling rendu (nvidia profiler, render doc)

## 6 Optimisaton Unreal

### 6.1 Profiling general

Unreal Engine propose de nombreux outils de proffiling, certains sont notamment accecbiles en tapant des ligne de commandes : stat FPS =j, affiche les fps.

Une commande qui permet de donner un aspect plus global est : stat unit, elle fournit les infos suivantes :

- Frame : temps pour rendre un image
- Game : temps de la partie gameplay
- Draw : temps pour afficher modeles
- GPU : Le temps que prend le GPU.
- Draws : Le nombre d'appels à DrawPrimitive
- Prims: Le nombre total de triangles rendus à l'écran

Plus une valeur est élevée plus elle est susceptible d'être cause de ralentissement.

## 6.2 Stat profiler

Le stat profiler appelé avec la commande : stat StartFile / stat StopFile permet de record des parties du gameplay pour y identifier les bottlenecks.

## 6.3 Profiler UE5 (Insight)

Quant à Unreal Insights il s'agit d'une version du profiler plus moderne est directement intégré dans l'éditeur (Tool > Profile > Animation Insights).

Pour s'en servir on lance l'éditeur en mode standalone game (de préférence sur une autre machine pour ne pas affecter les perf avec l'éditeur. On utilise ensuite une commande pour paramettrer le profiling, on fait -trace= suivi des parametres qui nous iterresent (CPU, GPU, Frame, LoadTime)

Dans la fenetre Unreal Insight on a accès à une timeline dans laquelle on pourra analyser les performances à chaque instant du jeu/de l'édition, dans notre cas on ne s'interesse qu'au temps de jeu. Dans celui ci on pourra constater eventuellement des pics, synonyme d'une baisse des fps, en cliquant dessus on pourra voir les causes de ces ralentissements. On va pouvoir donc zoomer avec la mollete dans la section correspondant à l'image sélectionnée, ici nous verrons quels opérations prennent le plus de temps, en en sélectionnant une on aura encore plus de details concernant des differents appels que cette fonction fait, de ce fait on va pouvoir efficacement cibler les bottleneck.

## 6.4 GPU Bottleneck

La commande ProfileGPU affiche une fenetre qui donne le call des fonctions GPU en temps réel, elle permet d'avoir une idée des fonctions qui prennent le plus de temps à s'exécuter afin d'aider à cibler le probleme de performance.

Le botleneck est gpu si GPU ou Draw sont en rouge, s'il s'agit de Draw la surcharge de polygones ou les textures trop grosses sont surement en cause, s'il s'agit de gpu il faut verifier les shader, le lightning ou les shadow.

Pour trouver la source du problème on utilise la commande stat scenerendering ou bien le view mode + arg pour des check plus ciblés.

## 6.5 CPU Bottleneck

La commande stat DumpFrame -ms=0.1 permet de lister dans le cmd le temps de call des fonctions CPU pour comprendre des pertes de perfs à un moment précis.

Si stat unit affiche un game rouge alors le bottleneck est CPU, il faut alors optimiser le code.

On évite de tout faire dans l'évent tick =i on y met juste à jour les variables. On privilégie des custom event. Des timeline peuvent faire l'effort. Les opérations mathématiques doivent plutot etre faite en c++.

Il faut faire tres attention à la partie physique, il ne faut pas tout rendre physique, éviter les mesh collider =i collider plus simples. Unreal permet donc de gérer deux versions de collider : Simple, Complex.

## 6.6 Bonnes pratiques

Enlever les Cast Shadows inutiles.

Downgrade les textures sur les petits objets.

Baisser le nombre de triangles sur les props peu visibles.

Désactiver la physique sur les objets qui ne l'utilisent pas.

Passer les objets en static.

Build les lights.

Régler les post process.

## 6.7 Multithreading

La gestion des thread peut etre une variable importante à prendre en compte dans le travail d'optimisation sur Unreal, de base le moteur se repose sur des thread principaux : Game Thread il gère la logique UObject, Render Thread fait les commandes de rendu, RHI envoie les commandes de rendu au GPU.

## **6.8 Techniques d’optimisations**

### **6.8.1 Frustum Culling**

Ne rends pas les objets en dehors du FOV de la caméra, automatiquement géré par le moteur.

### **6.8.2 Occlusion Culling**

Ne rends pas les objets invisibles du pdv de la caméra, peut être précalculé.

### **6.8.3 Distance Culling**

Ne rends pas les objets à une certaine distance.

### **6.8.4 LOD**

Permet de downgrade les textures et les meshes en qualité/précision pour alléger la charge de ceux ci, peut être géré automatiquement comme manuellement.

### **6.8.5 Level Streaming**

Le Level Streaming consiste en le chargement et le déchargement de parcelles de niveau afin d’alléger la force de calcul allouées à l’affichage et au chargement de données qui sont invisibles au joueur sur un instant t.

Pour le mettre en place on commence par fractionner notre niveau, on en garde une base qui sera persistante du début à la fin, tandis que les autres pourront être désactivés et activés à notre guise.

Pour accéder au flux de niveau on ouvre la fenêtre Levels, on crée des Levels sans World Partition, et on les drag and drop dans l’editor. Il est facile ensuite de charger et décharger les parcelles de niveau, pour ça on peut utiliser des trigger box, qui enclencheront des Load et Unload Stream level dans le level blueprint persistant.

### **6.8.6 Mise en cache PSO**

Ce dispositif permet de précharger les shaders en jeu une première fois et de les mettre en cache afin de soulager la charge de compilation de ceux ci sur les prochaines sessions de jeu.

### **6.8.7 Light complexity**

C’est en général la première chose qu’on regarde en cas de performance trop basse, pour ce faire on active le mode de rendu correspondant dans l’éditeur,

en général le fix sera de réajuster le rayon d'atténuation de la lumière sur nos mesh.

#### **6.8.8 Shader complexity**

De la même façon on active le mode de rendu, il faut que la scène soit verte au maximum, si le rendu de fenêtre etc sont en rouge ce n'est pas forcément problématique, au final la balance doit être trouvée entre la complexité du rendu et les performances. Si on se retrouve à vraiment devoir optimiser les shaders un choix possible serait de limiter le nombre de textures utilisées et privilégier des modifications de paramètres et de couleurs.

#### **6.8.9 Overdraw**

L'overdraw (quad ou nanite) survient quand trop d'éléments sont superposés au rendu proche du joueur, pour l'éviter on essaye de rassembler les éléments rassemblables dans la mesure du possible. Éviter le feuillage maské en nanite. Utiliser les lod.

### **6.9 Render Doc**

Avec la commande render doc capture frame on peut ouvrir l'outil render doc qui permet d'analyser en détail une frame de rendu. On a notamment une vue sur la durée de toutes les informations GPU.

RenderDoc permet de capturer l'état exact du pipeline GPU, draw call par draw call, avec accès aux shaders, buffers, textures et passes.

Pour s'en servir il est important d'activer le plugin du même nom dans Unreal